The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Appendix

C

Answers to the Checkpoints

Taught in —

Laboratory —

Assessment —

This is a class extract. It is *Appendix C* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter C

Answers to the Checkpoints

Three answers per chapter, in order. Attempt the checkpoint from memory before reading these, and argue with them where you disagree — several are judgement calls rather than facts, and they say so.

Chapter 1 — What E-Commerce Is, and What It Changed

1. **Electronic business, not electronic commerce.** Viewing results and downloading transcripts are business processes conducted over a network, but no value changes hands through the system — the transaction happens at a bank counter. The moment fee payment moves online it becomes e-commerce, and the university acquires every problem to the right of the dashed line in Figure 1.1: a payment that may have succeeded while the network dropped, a receipt that must exist exactly once, a refund path.

2. Many are acceptable. One good answer: **refusing to sell the last item when it is visibly damaged.** In software this requires a stock record that distinguishes "held" from "sellable", somebody whose job is to update it, a rule for what the storefront does when the two disagree, and a decision about what the customer sees — out of stock, or a quiet substitution? The shopkeeper does all of that in half a second without recording that a decision was made, which is exactly why it is forgotten when the shop goes online.

3. **Two uncounted costs:** the acquisition cost of finding customers who currently arrive free (Chapter 2 puts this at Rs. 1,200 an order for a typical small shop, which can exceed the commission), and the labour of running the four functions themselves — payment integration, delivery arrangements, customer service, returns. The commission was buying those, not just the listing. **What they gain that is not money:** the customer relationship. They learn who bought, can contact them again, and cannot be delisted. For a business with repeat custom that is frequently worth more than the 15%.

Chapter 2 — Business Models and Revenue Models

1. **Business model:** a marketplace connecting restaurants with diners, with fulfilment performed in-house — three and a half of the four functions. **Revenue models: three at once** — *commission* from restaurants, a *service or delivery fee* from customers, and *advertising* in the form of promoted placement. **Most profitable per unit: the advertising.** It has almost no marginal cost, where commission is earned against riders and support that must be paid for, and the delivery fee rarely covers the rider. Promoted placement is close to pure margin, which is why these platforms push it so hard.

2. Viable when LTV $>$ CAC, that is when $400 \times n > 1{,}500$, so when the average customer orders $n > 3.75$ — **at least four times** — before they stop. *What to measure:* the repeat rate and the time over which it happens. Four orders over six months is a business; four orders over six years is not, because the cash was spent today and returns over a period during which the venture has to survive.

3. B2C platforms store **one price per product** — a column on the product row. B2B pricing is a *relation* between customer and product, often with quantity breaks and expiry dates. That is not a setting you have failed to find; it is a different cardinality, and making it work means either a different platform or an extension that intercepts every price lookup in the system

(Chapter 19 shows how much work that is). The assumption that breaks is "a product has *a* price".

Chapter 3 — Auctions, Portals and Communities

1. **Bid Rs. 15,000** — your true valuation. You win, and you pay the second-highest bid, **Rs. 13,500**, gaining Rs. 1,500. Bidding Rs. 18,000 does not improve this outcome at all: you still pay Rs. 13,500, because the price is set by the others. What it does is expose you to winning when somebody bids Rs. 16,500, at which point you pay Rs. 16,500 for a thing worth Rs. 15,000 to you. The overbid can only ever buy you an auction you did not want.

2. It **never closes**. The auction stays open indefinitely; the winning bidder is never notified, the seller is never paid, and the listing accumulates further bids whenever somebody wanders past. The seller sees an item that appears to be permanently "ending soon" and a buyer who has given up. Worse, the failure is *silent* and inversely proportional to interest: popular auctions close correctly and unpopular ones do not, so the bug is invisible in testing, where you load every page you create.

3. Any two of: **be a seller yourself** until real ones arrive (cost: you take on stock and inventory risk, and you compete with the sellers you are recruiting); **narrow the market** to one city or one category (cost: you cap your growth and may pick the wrong niche); **give sellers a tool worth having without buyers** (cost: you are now building and supporting a second product); **list supply gathered elsewhere** (cost: legally risky and reputationally expensive if discovered).

Chapter 4 — Planning an E-Commerce Framework

1. (a) **Functional**, and it *does not* discriminate — all four options reset passwords. (b) **Functional**, and it **discriminates sharply**: per-customer pricing eliminates most hosted platforms and the marketplace route outright, and is the single most common reason to choose self-hosted or bespoke. (c) **Non-functional**, and it discriminates *in the opposite direction* to what students expect — surviving a twentyfold spike is something a large hosted platform does effortlessly and your single virtual server does not.

2. **Hosting** (Rs. 324,000 in the worked example), **setup and theming labour** (Rs. 175,000 plus Rs. 100,000 of custom modules), and above all **maintenance labour** (Rs. 360,000). Their stated saving of Rs. 792,000 is therefore closer to Rs. 339,000 — still a saving, but less than half what they claimed, and it vanishes entirely if maintenance takes eight hours a month instead of four.

3. Many are acceptable. A clear one: **a home baker selling six kinds of cake to one city, taking orders through Instagram.** Self-hosting would mean paying for a server and maintaining it to serve perhaps thirty orders a week, with nobody to patch it — which is not a cheaper shop but an unpatched one. **Choose marketplace-only or a cheap hosted platform**: the business's scarce resource is the baker's time, the volume is low, nothing about the pricing is unusual, and every hour spent on a server is an hour not spent baking.

Chapter 5 — The Stack, and Your Shop in One Command

1. **Working:** the operating system and the web server — a 500 is generated and served by Apache, so the request reached it and a response came back. **Failed:** the application layer; PHP raised an error and died before producing a page. The database is *unknown* — it may be the cause (the application could not connect) or entirely healthy. **Look first** at the PHP error log, with `docker compose logs shop`; the message is never on the screen, because `display_errors` is off, and Chapter 16 is about why you want it that way.

2. `depends_on` orders container *starts*, and a container that has started is not a database that will accept a login — MariaDB opens its TCP port several seconds before it finishes initialising. The shop therefore starts, runs the installer, and the installer is refused by a database that is listening but not ready. **The failure is intermittent and worse on a fast machine**, because the faster the shop container starts, the more certainly it arrives early. The `service_healthy` condition waits for the database's own healthcheck to pass, which tests a login rather than a port.

3. **Lost:** both named volumes — the entire database (products, categories, orders, customers, settings) and the shop's installed files, including any module you had written in place. **Not lost:** anything in the repository — the Compose file, the Dockerfile, `code/`, and any module kept there rather than edited inside the container. **What would have made it minor:** a database dump taken on a schedule and kept outside the volume, plus the discipline of writing modules in `code/modules/` and mounting them, so that the only thing destroyed is state you can regenerate. Chapter 20 makes both of these routine.

Chapter 6 — Products, Categories and Their Tables

1. **Move to `oc_product_description`:** the product name and the meta description. Both are *words*, and words differ by language. **Stay in `oc_product`:** price, weight and stock quantity — a kilogram is a kilogram in Urdu, and a shop that let the price depend on the reader's language would eventually charge two different prices for one product. The test is the first of the four questions: *does this fact depend on who is reading?*

2. **Buys:** a single-query answer to "every category beneath this one", so a category page costs one query instead of a loop whose length depends on how deep the tree happens to be. **Costs:** the same fact is now stored twice, so the two can disagree, and every write that changes the shape of the tree must rewrite the derived rows. **The operation that makes it visible:** *moving a category.* Changing one `parent_id` invalidates the path rows of that category and of everything beneath it, so the admin panel must rewrite them all — which is why moving a category with many descendants is noticeably slower than renaming one.

3. Two of several good answers. **(a) The database was never the bottleneck**: if the database is 20% of the page, halving it improves the page by 10%, which can be smaller than the run-to-run variance. **(b) The expensive thing is the *number* of queries, not the cost of any one** — the measurement in this chapter found 146 queries for one page, and an index helps exactly one of them. **To tell them apart:** turn on the query log with `long_query_time=0`, request the page once, and read `COUNT(*)` and `SUM(query_time)` from `mysql.slow_log`. A large total with few queries is (a); a small total spread over a hundred queries is (b). That single measurement separates them, and it is the one in `m06_catalogue.py`.

Chapter 7 — Product Variations, Options and Uploads

1. **The photograph is an option** — specifically a `file` option. It is supplied by the customer, it is different every time, and the shop holds no stock of it. **The size is a variant**, because a small mug and a large mug are two different objects on two different shelves, each with its own cost, its own barcode and its own count. **What the shop holds in stock:** blank mugs in two sizes. Nothing else — which is exactly why the photograph cannot be a variant: there is no such thing as stock of "the mug with Ahmed's photograph on it" until Ahmed orders one.

2. **Option value rows:** $4 + 3 + 2 = \mathbf{9}$, plus three rows in `oc_product_option`. **As variants:** $4 \times 3 \times 2 = 24$ **combinations, so 25 `oc_product` rows** — twenty-four plus the master — and 25 description rows, 25 store rows and 25 category rows beside them. Nine against a hundred, for the same catalogue.

3. **Passes:** the filename sanitiser (nothing illegal in "logo.svg"), the length check, the extension allow-list (`svg` is on the stock list), and the MIME allow-list — it claimed `image/png`, which is allowed, and nothing verifies that claim. **Fails:** nothing. All five checks pass. **For it to harm a visitor**, the file must be *served to them from your origin and rendered*. Two things would have to be true: the attacker must learn the stored URL, which carries a 32-character random token and so is not guessable — but it *is* shown to whoever uploaded it and may be forwarded to an administrator who opens it — and the server must serve it with `Content-Type: image/svg+xml` rather than as a download, which Chapter 5's stock install does. The realistic victim is therefore the administrator reviewing the order, and the realistic fix is to drop `svg` from the list and serve uploads with `Content-Disposition: attachment` (Chapter 16).

Chapter 8 — Themes, Layout and the Customer's Experience

1. **Best case for compressing the photographs:** the images are 713 − 597 − 54 = about **61 KB**. Halving them saves 30 KB, which is **4%** of the page — and that is the *best* case, because you cannot compress them to nothing. **Propose instead:** minify `bootstrap.css`, which is 265 KB of the 597 and has no minified copy in the tree. That single change saves roughly 240 KB, about **34%** of the page — eight times the whole image budget — and changes nothing a customer can see. Then remove the unused parts of Bootstrap and FontAwesome, and only then look at the images.

2. **Layout change** if a "trust badge" module already exists: **Design › Layouts › Product**, add the module to **Content Bottom**, set its sort order. No code, visible to the next person, survives upgrades. **Template override** if it must sit *immediately under the button*, inside the markup the theme generates: there is no position there, so you override `product/product.twig` in your child theme and add it. The general rule: a position that exists is a layout change; markup that must differ is a template change.

3. **Two reasons it is lower.** (a) **It follows only assets named in the HTML.** Fonts are requested by the CSS once the browser parses it, so none of the FontAwesome `.woff2` files — 248 KB in that directory — are counted. (b) **It does not execute JavaScript**, so anything a script loads afterwards (a tracking pixel, a lazy-loaded image, a carousel's later slides) is invisible to it. **Which it could fix:** the first, by parsing the CSS for `url()` references and following them — a dozen lines. The second it cannot fix, because fixing it means running a browser, which is a different kind of tool. Knowing which limitation is laziness and which is fundamental is the useful judgement here.

Chapter 9 — The Shopping Basket, and Where State Lives

1. **What would change:** the two queries per request that the `db` engine spends reading and writing the session record. On this host that was worth about 20% of the page — 89 ms to 71 ms — when those two queries were removed altogether by the `file` engine, and Redis would remove them from the database in the same way. **What would not change: the basket.** It is rows in `oc_cart`, and the `Cart rows` column was 1 under both engines. **The number that settles it:** the session record is **18 bytes** and holds `{"currency":"USD"}`. Whatever you move it to, you are moving eighteen bytes. The right place to look for basket performance is the six cart queries an empty basket costs, not the store the session sits in.

2. **Where they came from:** a session row is written on every request, before the visitor has asked for anything, so every page view by every crawler, uptime check and measurement harness created one. The 645 rows here are the by-product of writing this handout. **Why they are not removed on departure:** nothing tells a server that a visitor has left — HTTP has no "goodbye". Rows can therefore only expire on a clock, and `config_session_expire` is 86,400 seconds. **What to measure first:** the proportion of sessions that ever acquire a basket

or a login. If almost none do, the expiry is not the problem and robots are; shortening the expiry would then also shorten the life of the few baskets that matter, which is the opposite of what was wanted. Measure before tuning the number that is easiest to reach.

3. **What each removes:** `down` removes the containers — the running processes and their writable layers. The named volumes `dbdata` and `shopdata` are separate objects and are left alone, so the database files survive and the basket with them; `up -d` attaches the new containers to the same volumes and the shop resumes mid-visit. `down -v` additionally deletes those volumes, which is where every byte of durable state lives, so the entrypoint finds an empty database and reinstalls from scratch. **One other thing it destroys:** every product, category, order, customer and setting you have made since Chapter 5 — in this measurement the shop came back in 17 seconds looking perfectly healthy, with all of it gone. Uploaded images go with the `shopdata` volume at the same time.

Chapter 10 — The Checkout and the Order State Machine

1. **Three things, in order.** (a) **Drive the checkout end to end from a script and read every reply**, not the rendered pages. (b) **Find the first step whose answer disagrees with the step before it** — here, an address accepted with `success` and then reported missing. (c) **Read the source for that one step**, which the previous two have now identified out of several thousand files. **What the measurement would have told you:** that the shipping quote answered `"Shipping address required!"` — a fault, not a price objection. The shipping charges were never the question, and a shop that had lowered them would have changed nothing and concluded that customers were simply difficult.

2. **Why the shares differ.** Time is spread across all eight steps because every one of them loads the framework, starts a session, rebuilds the basket and renders something — fixed costs that have nothing to do with what the step is for. Writes are not spread: seven steps write almost nothing, because collecting an address changes a session, while `confirm` commits the whole order — `oc_order`, its products, its totals and its addresses — in one burst. **What it implies at ten times the traffic:** optimise the *fixed* cost, not `confirm`. Reads scale by caching and by the work of Chapter 6; writes do not cache, but 55 writes per order against 607 queries means the order itself is the small part. The 271 queries spent before the customer has committed to anything are the ones to attack.

3. **Two reasons to refuse.** (a) **They are evidence.** Each one is a customer who reached the payment button and stopped; the ratio of status-0 rows to real orders *is* the abandonment rate, and deleting them destroys the only record of it the shop has. (b) **A payment provider may still be looking for one.** A gateway's reply can arrive minutes or hours later, and it finds the order by an identifier that must still exist (Chapter 14); deleting the row turns a late but successful payment into money received against nothing. **What to build instead:** a report that counts them by day and by last-reached step, and an archiving rule that moves rows older than the gateway's longest callback window — so that nothing is destroyed and the live table stays small.

Chapter 11 — Shipping, Tax and Zones

1. **Where the 141.40 comes from.** It is not 20% of the sub-total: $0.20 \times 702.00 = 140.40$, which is 1.00 short. The missing 1.00 is 20% of the 5.00 shipping charge, so the base is $702.00 + 5.00 = 707.00$ and $0.20 \times 707.00 = 141.40$. **The two rule decisions this reveals.** (a) **Shipping is taxable** in this configuration — a jurisdictional choice, not arithmetic, and one that differs between countries. (b) **The fixed eco tax is not itself taxed**: it contributed 6.00 to the total but nothing to the VAT base, so the two tax lines are applied to the same

base in parallel rather than one on top of the other. Had they compounded, the VAT would have been $0.20 \times 713.00 = 142.60$.

2. **What is wrong.** A base of `store` means every order is taxed as though it were delivered to the shop's own address in Lahore. **Which orders are affected:** only the UAE ones — they are charged Pakistani sales tax on an export that should not carry it, and the shop is either over-collecting from the customer or absorbing the amount itself, while the correct treatment is recorded nowhere. Every domestic order is, by coincidence, correct. **Why nobody has noticed:** the configuration is right for the overwhelming majority of orders and for every order anyone has ever tested, because testing is done from Pakistan. A fault that is correct in the common case and wrong in the rare one does not announce itself. **What to change:** set the rule's base to `shipping`, and confine the rate's geo zone to Pakistan, so that a UAE delivery matches no rule and attracts no Pakistani tax. Then place an order to each, because a change made on reasoning alone has not been verified.

3. **The failure mode in money.** Weight-based shipping charges from a band table keyed on total basket weight. A product with weight zero contributes nothing, so a basket of only zero-weight products falls in the lowest band — usually free or near-free delivery — and the shop pays the courier out of its own margin. With 600 of 4000 products affected, roughly one basket in seven will be understated, and the worst cases are the heaviest products whose weight was never entered. **Why no error appears:** zero is a valid weight. The band lookup succeeds, a quote is returned, the order completes, and every page returns 200 — the same shape of silent failure as Chapter 10's missing setting. **One change that makes it visible:** refuse to quote for a basket containing a zero-weight product, or, less disruptively, add a dashboard panel (Chapter 17) counting enabled products with a weight of zero. The first prevents the loss; the second reports it. Either is better than an accurate method nobody maintains.

Chapter 12 — Discounts, Vouchers and Referrals

1. **The setting** was the coupon total extension's *sort order*: 4 as shipped, which places it before the tax line at 5, versus 6, which places it after. **Why 14.04:** the discount is 70.20, and the VAT rate is 20%. With the coupon first, the VAT base is $702.00 + 5.00 - 70.20 = 636.80$ and the VAT is 127.36; with it last, the base is $702.00 + 5.00 = 707.00$ and the VAT is 141.40. The difference $141.40 - 127.36 = 14.04$ is exactly 0.20×70.20 — tax on the discount. **Which is correct: 770.16**, the shipped order. Tax is owed on *consideration* — what the customer actually pays — and the customer was never asked for the 70.20, so no tax is due on it. Charging 784.20 collects 14.04 of tax on money that changed hands with nobody.

2. **The break-even uplift.** With margin $m = 0.25$ and discount $d = 0.15$, the multiple is $m/(m-d) = 0.25/0.10 = \mathbf{2.5}$ — the shop needs **150% more orders**, two and a half times the volume, to make the same profit. **Two reasons the real requirement is higher.** (a) **Cannibalisation**: a permanent code for *loyal* customers goes, by definition, to people who already buy. Every one of them who would have bought anyway is pure loss, and no new volume is created to offset it. (b) **Variable costs scale with volume**: 150% more orders means 150% more picking, packing, delivery and support, so the gross-margin arithmetic above understates the cost of getting there — and a permanent code also resets the reference price, so the 15% is not recoverable later. **A cheaper alternative:** store credit issued *after* an order — say 5% back, usable on the next purchase. It costs less (not every credit is redeemed), it cannot be found by someone searching for a code, it rewards repeat buying rather than discounting a single sale, and it brings the customer back rather than merely letting them leave cheaper.

3. **What to tell the client.** That the platform has no gift vouchers at all — no tables, no totals extension, only a leftover demonstration image from the version that did — and that the outline naming them reflects a 2010 book rather than current software. Say it before quoting,

not after. **What the platform already provides:** the *spending* half. Store credit (`oc_customer_transaction` and the `credit` total) is a balance on an account that draws down across orders, handles partial payment, and already sits in the totals pipeline. **What must be built:** the *buying* half — selling a voucher as a purchasable product; generating a code on order completion; delivering it to a named recipient; and redeeming it into *a different customer's* credit, with the balance, expiry and single-use guarantees that implies. That is a module (Chapter 19), and the hard parts are the two ways it can be spent twice: two simultaneous redemptions of the same code, and a redemption on an order that is later refunded.

Chapter 13 — Web Payment Systems

1. **Two ways to ship goods you are never paid for.** (a) **Authorisation mistaken for capture.** The gateway reserved the funds and answered "success", but nothing was ever claimed. The reservation lapses after a few days and the money returns to the customer, who has the goods. (b) **Capture followed by a chargeback.** The money arrived and was later taken back by the issuer — on a disputed or fraudulent card, this is the shop's loss, not the acquirer's. **What to check in the documentation:** for the first, whether the endpoint you call performs authorisation only or authorisation-and-capture, and if the former, which call captures and when it must happen. For the second, the provider's chargeback terms: how long the window is, what evidence wins a dispute, and whether a delivery signature is among it. Both questions have documented answers, and neither is visible in the integration's happy path.

2. **Net margin at a 10% refusal rate.** Per 100 orders: 90 delivered at $900 - 60 - 250 =$ Rs 590 each, so $90 \times 590 =$ Rs 53,100; 10 refused at −Rs 500 each, so −Rs 5,000. **Net Rs 48,100**, against Rs 37,200 at a 20% refusal rate — an improvement of Rs 10,900, or **29%**. **Against removing the courier fee entirely:** the fee is Rs 60 on each of 80 delivered orders, $80 \times 60 =$ Rs 4,800, which raises the net to Rs 42,000 — an improvement of **13%**. So halving the refusal rate is worth more than twice as much as winning a free COD service, and the courier would never agree to the second while the first is entirely within the shop's own control. **The lesson:** negotiating hard over the thing you cannot change while ignoring the thing you can is exactly backwards, and the arithmetic says so in one line.

3. **Four things, in order.** (1) **Can you legally receive the money?** Foreign-currency receipts by a Pakistani merchant are subject to State Bank rules, and the answer determines whether the project is possible at all. (2) **Will an acquirer accept this business?** Cross-border card acceptance is underwritten against the risk that the shop does not deliver, and a new shop with no trading history is often declined. (3) **What does it cost, including the dispute rate?** International card-not-present carries both a higher discount rate and a materially higher chargeback rate than domestic. (4) **What will you ship, and how will you prove you shipped it?** Winning a dispute requires delivery evidence, and international delivery often produces none. **Most likely to stop the project: the second.** The technical integration is a week; acquirer onboarding for cross-border acceptance by a small Pakistani merchant is the step that fails — and it fails after everything has been built, if you leave it until last. That is why it belongs second on this list and not fourth.

Chapter 14 — Taking Payment for Orders

1. **What a 1-second timeout actually costs.** From the measured table it cuts off **23 of 196 answered requests, 11.7%** — not a few stragglers but one payment in eight and a half, each with an *unknown* outcome rather than a failed one, so the customer may already have been charged. The colleague's reasoning is sound for a bell-shaped distribution, and this one is not bell-shaped: the median is 321 ms and the 95th percentile is 3,774 ms, so "three times the median" lands in the empty gap between the two groups. **What to propose: 5 seconds**,

which cut off nothing at all, and the measurement to justify it. The customer waits longer in the rare slow case, and the shop stops discarding good payments. If five seconds is judged too long to make anyone wait, the answer is not a shorter timeout but an asynchronous design — accept, show a pending page, and resolve from the provider's callback.

2. **Why the middle is flat.** Because the latency distribution is *bimodal*: one group answers in under about half a second and a second group takes between two and five seconds, with very little in between. A timeout placed in the gap therefore catches almost nothing extra — going from 1 s to 3 s rescued two requests in two hundred. **What the shape tells you that an average could not:** that there are two distinct code paths inside the provider rather than one variable one — a fast path, and something slower behind it such as a risk check or an internal retry — and that the only meaningful choices are "before the slow path" and "after it". An average of 700 ms invites tuning around 700 ms, which is precisely the region where no choice makes any difference at all.

3. **Without a key.** The second request is a new, valid authorisation. The provider has no way to tell it from a customer legitimately buying twice, so it charges again and returns a different transaction identifier — in the measurement `f59594f7d23c4446` and `8171b6e85f9b4766`, both approved. The customer has paid twice and the shop has one order. **With a key.** The provider recognises it, returns the first answer unchanged — same transaction identifier, in 5 ms — and charges nothing. **Where it is stored: on the order row**, generated once when the order is created. **Why generating it when the button is pressed would not work:** the retry is a new request and would generate a new key, which is a key the provider has never seen, so it charges again. The key must identify *the payment attempt for this order*, not the HTTP request — and the order is the only thing that survives between the two.

Chapter 15 — User Account Management

1. **Is cost 12 affordable?** Assumptions, stated: 400 orders a day; perhaps three logins per order once failed attempts, repeat visits and account pages are counted, so about 1,200 logins a day; and traffic is not flat, so assume the busiest hour carries 20% of the day's logins — 240 in an hour, about **0.07 logins per second** on average and perhaps one per second in a burst. Cost 12 delivers **5 logins per second** on one core. The headroom is more than fiftyfold, so **yes, comfortably** — and cost 13, at 2 per second, would still be adequate. **What else must change at the same time:** the timing leak. Cost 12 raises the enumeration gap from about 65 ms to about 220 ms, four times louder, so the dummy-hash fix must ship *with* the cost change rather than after it. Existing hashes also remain at cost 10 until each customer next logs in, so the code must rehash on successful login or the change never reaches anybody.

2. **The cause.** A wrong password on a registered address must be checked against the stored hash, which costs one bcrypt verify — measured at 60.3 ms at cost 10. An unknown address matches no row, so there is nothing to verify and the request returns early. The 65.5 ms gap is that verify. **What an attacker does with it:** runs a list of email addresses through the login form with any password at all, and keeps the slow ones. That is a confirmed customer list — saleable in itself, and it makes credential stuffing and phishing against this shop's customers far cheaper, because every address is known to be live. **The one-line fix:** when no customer matches, verify the supplied password against a fixed dummy hash and discard the result. **Why it feels wrong to write:** it is deliberately wasted work in the hot path of a login form, and every instinct a programmer has says to delete it. A reviewer who does not know why it is there will delete it, which is exactly why the reason belongs in a comment beside it.

3. **The risk you are accepting** by removing lockout: unlimited online password guessing against any account. At 60 ms per attempt a single attacker can try tens of thousands of passwords an hour against one account, which breaks any password a customer actually chooses. **The risk**

you are removing: anyone who knows a customer's email address can lock that customer out at will — five wrong guesses, an hour at a time, repeatedly — so the complaint is real, and the customers complaining may be victims rather than the forgetful. **A design that addresses both better than either:** replace the hard count with *progressive delay* keyed on the address being guessed — no delay for the first three attempts, then a delay that doubles with each further failure and resets on success. A customer who mistypes twice notices nothing; an attacker is throttled to a few guesses an hour, and no account is ever locked, so there is nothing to abuse. Add site-wide rate limiting by source address to catch the attacker who spreads attempts across many accounts — which neither lockout nor per-account delay sees at all — and offer a second factor to the customers who want one.

Chapter 16 — Securing the Shop

1. **Two independent reasons.** (a) **Apache never reads the file.** Debian's `apache2.conf` sets `AllowOverride None` for the document root, so `.htaccess` files are ignored entirely — the rename is correct and has no effect. (b) **Even read, it would not be the whole fix.** The shipped file blocks `system/storage` by rewrite and sets one header; it says nothing about the content security policy, the referrer policy, the permissions policy, the version banners or the wildcard CORS headers, which were eight of the nine findings. **One command to demonstrate it:** write a marker into the error log and fetch it — `docker compose exec shop sh -c 'echo MARKER » system/storage/logs/error.log'` then `curl -s http://localhost:8090/system/storage/logs/error.log`. If the marker comes back, the control is not working whatever the file is called. **The general lesson:** a control that fails open and fails silently is indistinguishable from a control that works, until somebody measures the outcome. Verify the result, never the action — and the gap between "I installed it" and "it does something" is where almost every real breach lives.

2. **Why none of them touched the shop's code.** Because none of them was a programming fault. Every one was a property of how the software was *served*: which headers Apache attaches, what it will serve from which directory, and what it volunteers about itself. The application had already escaped its SQL, auto-escaped its output, tokenised its forms and hashed its passwords properly. **What that says about inheriting a shop:** look at the deployment first. The framework's defences are written once by people who do this full time and are the same in every installation; the deployment is different in every installation and is usually nobody's job. **Which remaining finding first: move `system/storage/` out of the web root.** The admin path is the one still failing the battery, but renaming it only defeats automated scanners, while the storage directory holds the error log, the sessions and every upload — and today it is protected only by a block in a configuration file that the next administrator may edit, replace or lose during an upgrade. Moving it removes the risk instead of covering it, and is the control that survives being forgotten.

3. **Two mechanisms, any two of the three.** (a) **`Header always unset` on a successful response.** `mod_headers` keeps two tables — `always` for every response including errors, and the default for successful ones only. A header PHP sets on a 200 lives in the second, so the directive whose name sounds more thorough touches nothing. (b) **`ServerTokens` after a reload.** It is consumed when Apache starts and used to build the banner once; a graceful reload applies every neighbouring directive and skips this one. (c) **Filename ordering in `conf-enabled`.** Apache reads the directory in filename order and the last setting of a server-level directive wins, so `security-ecad.conf` lost to `security.conf` because `-` sorts before `.`. **The habit that catches all three without knowing any of them:** *measure the observable outcome before and after every change, from outside the system.* Not "is the directive present", not "did Apache accept the config" — all three of these produced a configuration Apache accepted without a single warning — but "does the response now contain the header, and does

the URL now refuse to serve the file". One script, run twice, found all three in the time it would have taken to re-read the file once. That is the entire argument for the measured strand in this course, in its most concrete form.

Chapter 17 — Administration: The Dashboard, and What to Watch

1. **Each gap. 6 to 5:** the table counts every row, including the one at status 0 — an order created at `confirm` whose customer never paid (Chapter 10). That difference is the *abandonment*. **5 to 2:** the widgets count every status above 0, while the chart counts only those in `config_complete_status`, here Complete and Shipped. The three missing orders are paid for and not yet despatched — the *work outstanding*. **Which figure to put in front of the owner: 5, labelled "orders to fulfil and bank"**, because it is the number the shop will actually be paid for. But put it beside the other two rather than instead of them, each with its filter named: "5 paid, 3 still to despatch, 1 abandoned today". One number with a label beats three without, and the labels are the whole content.

2. **Questions to ask.** (a) **What is the denominator?** Sessions, unique visitors, people, or baskets created? Each gives a different answer from the same shop, and Chapter 9 showed that sessions are created by robots that never see a page. (b) **What counts as a conversion?** An order row, an order above status 0, or an order that was actually paid and not refunded? (c) **Over what period, and does it include the campaign spike?** (d) **Are bots excluded, and how? Why two shops at 3% may not be comparable:** one may divide paid orders by human visitors and the other all order rows by all sessions. With 645 session rows and no human customers at all — this shop's measured state — the second definition can report a rate that is pure arithmetic about crawlers. Comparing the numbers without comparing the definitions compares nothing.

3. **A panel that would have caught it. The query:** count orders created in the last hour, split by whether they are above status 0:

   ```
   SELECT SUM(order_status_id=0) stalled,
   SUM(order_status_id>0) completed
   FROM oc_order
   WHERE date_added > NOW() - INTERVAL 1 HOUR;
   ```

   How often: hourly during trading hours. **The alarm condition:** *completed* is zero while the shop has had traffic. That is the exact signature of Chapter 10's fault — customers arriving, baskets filling, no order ever reaching status 1 — and it is detectable without knowing anything about the cause. Note what makes it work: it alarms on the *absence* of a thing. Every monitoring system in the course so far watched for errors, and this fault produced none; the only observable was that something which should happen had stopped happening.

Chapter 18 — Managing Orders, Customers and Refunds

1. **Pending to Shipped.** Nothing happened to stock. Both statuses are inside the processing-or-complete set, and the rule fires only on entering or leaving it — the goods had already been deducted when the order reached Pending. **Shipped to Reversed.** Also nothing, which is the surprise: 12 is in `config_processing_status` in this shop, so the order never left the set. The customer's money has gone back and the goods are still counted as sold. **What to change:** remove 12 from `config_processing_status` at **System › Settings › Edit › Option**. Then moving to Reversed leaves the set and returns the stock, exactly as Refunded does. Note what this implies: the stock behaviour of every status in the shop is decided by two checkbox lists, and nothing in the interface connects them to the word "stock".

2. **The mechanism.** Marking a return **Refunded** writes two rows — one updating `oc_return`, one appending to `oc_return_history` — and touches nothing else. The goods physically come back to the shelf, but `oc_product.quantity` was never incremented, because the *order* was never moved out of the processing set. So after 40 returns a month the system believes it holds 40 fewer units than it does, every month, cumulatively. **What else is drifting:** the revenue figures, because the orders still read as sold and the dashboard counts them (Chapter 17); any affiliate commission paid on them, which nothing reverses (Chapter 12); and the shop's record against the payment provider's, which is what Chapter 14's reconciliation exists to catch. **The smallest change that stops it:** make moving the *order* to status 11 a required step of recording a return. One line in the procedure, and it fixes stock and revenue together, because both follow from the same transition.

3. **Four steps.** (1) **Reproduce it outside the browser** with a signed API call, which separates "the admin panel is broken" from "the API refuses". (2) **Read the response**: a 403 with a permission message, not a 500 and not a validation error. (3) **Check `oc_api`** — a user exists and is enabled, so it is not a missing credential. (4) **Check `oc_api_ip`** — empty, and an empty allow-list permits nothing. **Why it reads as a user permissions problem:** the error message is the generic "You do not have permission to access this page", which is the same text an administrator sees when their own user group lacks a privilege. So the natural first move is to inspect the administrator's permissions at **System › Users › User Groups**, find them correct, grant more anyway, and still fail — while the refusal is happening in a different application, to a different identity, over a connection the administrator never sees. The lesson is the one from Chapter 16: an error message tells you what the system decided, not where it decided it.

Chapter 19 — Extending the Platform: Your First Module

1. **The answer.** Five instances cost 20 ms on a 76 ms page — about 4 ms and exactly 4 queries each — and of those four queries *one* is the module's own. The other three are the framework finding the instance's settings, its translations and its template override. So the measured cost of "being a module" is 3 queries, and the cost of the feature is 1. **Editing the theme instead** would remove those three and keep the one, saving roughly 3 ms per instance — at the price of a change the next upgrade overwrites, that `git status` will not distinguish from the vendor's code, and that cannot be placed twice or disabled without another edit. **What to measure before agreeing to anything:** whether the page is actually too slow, and against what target. A 96 ms page with five modules is not a performance problem — Chapter 8 measured 600 KB of assets after it and Chapter 20 measured the saturation point at five concurrent clients. Optimising 3 ms while 600 KB goes unexamined is the error this whole course is written against.

2. **When it stops working.** The moment the module is placed a *second* time with a different threshold. Both instances will then read the same global value, so both show the same products, and whichever instance was configured last silently wins — or, if no such global setting exists at all, both read null and fall back to the default, so neither reflects what was typed. **What the shopkeeper will report:** "I set the second panel to 20 and it still shows the same four items as the first one." They will not report a bug in the module; they will report that the admin panel does not save. That is the characteristic shape of this fault — the symptom points at the form, and the cause is in the render method, which is the one place nobody looks because it is visibly working.

3. **How to decide.** First ask whether a *setting* reaches it: a custom field on the customer or the address, or a product attribute, covers more requests of this kind than people expect, and a setting survives every upgrade. If not, ask whether the platform offers an *extension point* at the place you need — for checkout there are events on the controllers, and an event handler

is an extension. Only if neither reaches do you consider editing, and then the question is whether the field is worth owning a merge conflict for ever. **What I would need to know first:** who fills the field in and who reads it; whether it must appear on the *order* as well as in the session, because an order copies its data at checkout (Chapter 10) and a field that is not copied disappears from history; and whether it is required, because a required field changes the validation path and therefore the completion rate. **What I would write down whichever I chose:** the decision, the two options rejected and why, and — if it was an edit — the file, the version it was made against, and what must be re-checked at the next upgrade. The note is the deliverable; the code is the easy part.

Chapter 20 — Deployment, Maintenance and Performance

1. **The mechanism.** At five concurrent clients the server's processors are fully occupied and nothing is waiting. Past that point the extra requests do not find spare capacity — they join a queue. A queued request still consumes memory, a connection and a worker slot, and the context switching between twenty workers costs real time, so the server does *less* useful work while every request waits longer. That is why throughput fell from 42.1 to 38.7 while the median rose from 95 ms to 455 ms. **What the owner should do before a sale day:** know the number, and decide what happens past it — queue politely with a wait page, or shed load — rather than discovering it at the busiest hour of the year. **One change that moves the turning point:** cache whole pages for visitors who are not logged in. It removes PHP and the database from the common path entirely, which is where every millisecond in the measurement was spent; the saturation point then becomes a property of the web server rather than of the application, and that is a different order of magnitude.

2. **Both ends. At one client** the processor is mostly idle: a request arrives, is compiled and served, and then nothing happens until the next one. Removing compilation work from a processor that has spare capacity buys little, so 1.3×. **At twenty** the bottleneck has moved from the processor to the *queue*: every request waits behind nineteen others, and shortening each one by a third shortens the queue proportionally but does not change the fact that there is one, so the benefit is diluted to 1.4×. **In the middle**, at the saturation point, the processor is the binding constraint and nothing else is — so work removed from it converts almost one-for-one into throughput, hence 2.0×. **What it implies about when to benchmark:** measure at or near the saturation point, because that is where an optimisation's effect is both largest and most honestly represented. Benchmarking a single client flatters nothing and shows nothing; benchmarking far past saturation measures the queue and attributes it to your change.

3. **The case, in terms a client will act on.** Not "your backups might be corrupt" — that is a hypothetical and clients discount hypotheticals. Instead: "We do not know how long the shop would be closed. I can find out this afternoon." Recovery time is a business number, it belongs in the same sentence as a day's takings, and nobody has ever been given it. **What I would do:** bring up a second empty stack, restore last night's dump into it, open the shop, and check that the catalogue, the orders and the settings are all there. **How long:** under an hour for this shop, including writing it down; the restore itself was 6.4 seconds on the demonstration catalogue and will be minutes on a real one. **What I would measure and record:** the restore time, the age of the newest data in the dump — which is how much the shop would lose, and is a different number from how long it is down — and anything that was *not* in the backup, which in this stack is the uploaded images in the `shopdata` volume. That last item is the one such a test usually finds, and finding it is the entire point.

Chapter 21 — SEO and Digital Marketing

1. **What has happened.** The client switched on `config_seo_url`, which is the *application* half of readable URLs: the shop now looks every link up in `oc_seo_url` and writes `/en-gb/catalog/desktops` into its HTML. The *server* half — the rewrite that turns that path back into `index.php?_route_=en-gb/catalog/desktops` before PHP sees it — is absent, because it ships in `.htaccess` and Chapter 16 measured `AllowOverride None`, which means Apache never reads `.htaccess` on this stack. **Which half is absent: the server half. The command that demonstrates it:** `curl -s -o /dev/null -w "%{http_code}" http://the-shop/en-gb/catalog/desktops` returns 404, while the same page via `index.php?_route_=...` returns 200 — which proves the application is right and the server was never told. **Why nothing appeared in any log:** a 404 is an ordinary response. It is not an error, not an exception, and not a failure of anything; the server was asked for a path it has no rule for and said so, correctly. The shop is working exactly as configured, and the configuration is wrong.

2. **Which is luck and which is design.** The distinct titles are **luck**: the title is the product's name, and the demonstration products happen to have different names. A shop selling the same garment in four sizes as four products would have four identical titles and the same code would produce them. The missing descriptions are **design** — or rather, the absence of design: the field exists in `oc_product_description`, the demonstration data leaves it empty, and the platform writes nothing in its place. **What happens on those pages:** the search engine composes its own summary from whatever text it finds, which on a product page is usually the navigation menu — so the snippet a customer reads in the results is a list of categories rather than a description of the product. **For 50,000 products:** do not write them by hand. Generate a reasonable default from data you already have — name, manufacturer, category and price — as a module (Chapter 19), which gets every page off the floor; then write proper descriptions by hand for the few hundred products that actually sell, because those are the only ones where the difference between adequate and good is worth an afternoon.

3. **What each absence costs. No sitemap** means a crawler finds pages only by following links from the home page, so anything several hops away is effectively invisible — and depth grows with catalogue size, since a 50,000-product shop paginates its categories into hundreds of pages that `robots.txt` itself blocks. **No canonical links** means the same product reached through two categories, or with a sort parameter, is several URLs with identical content competing with one another, and the engine picks one arbitrarily. **For 19 products the canonical matters more:** every page is one click from the home page so nothing is unfindable, but nineteen products across several categories already generate duplicate paths. **For 50,000 the sitemap matters more:** duplication is still a problem, but it is a problem about pages that are at least known to exist, and the greater loss by far is the tens of thousands that are never reached at all. **The difference:** canonical links are about which of the pages a crawler found should count; a sitemap is about whether it found them. Being counted wrongly is a smaller harm than not being counted.

Chapter 22 — Social, Legal and Ethical Issues

1. **Is it a dark pattern? Yes** — and the argument has to be made carefully, because every individual element is honest. The sub-total is right, the discount is right, the tax is arithmetically right on the base it is applied to, and the total adds up. What makes it a dark pattern is the *arrangement*: the customer is shown a discount of 70.20 and charged tax on money that discount means they never paid, and no part of the screen shows them that this is what happened. That is the definition — leading someone to something they would not have chosen, by how it is presented rather than by a false statement. **What would change my answer:** if the tax authority's own rule required tax on the undiscounted amount, the arrangement would

be compelled rather than chosen, and the shop would be a collector rather than a beneficiary. The test is who keeps the 1.8%. **What I would do in an inherited shop:** assume it was an accident, because Chapter 12 showed it is one integer in an unlabelled field and far more likely to be a mistake than a scheme. Fix it, then say what was fixed, how long it had been that way, and roughly what it came to — because if the shop has been over-collecting, somebody else's money is involved and that is now a question about refunds rather than about configuration.

2. **The ranking: the 6 orders are higher risk than the 645 sessions**, by a wide margin, and the volume points the other way. **Justification:** a session row holds an address, a timestamp and a currency for an unidentified visitor who was probably a crawler (Chapter 9); leaked, it is noise. An order row holds a named person's home address, telephone number, email address and a list of what they bought, delivered to where they live; leaked, six of them would do more harm than all 645 sessions together. Risk is harm per record times records, and the first term dominates by orders of magnitude. **Retention. Sessions: 24 hours**, which is already the configured expiry — they exist to make the next click work and have no purpose after the visit, so the only argument for keeping them is that nothing deletes them, which is not an argument. **Orders: years**, because they are accounting records with a statutory life and are the contract (Chapter 22's opening). The *personal* parts of an old order may be reducible — a telephone number is not needed to satisfy a tax inspector — but that is a decision to take advice on, not one to take alone. **Which I would be more reluctant to delete: the orders**, and the reason is the opposite of the risk ranking. They are the most dangerous thing the shop holds *and* the thing it is least free to discard, which is exactly why they deserve the most protection rather than the most deletion. The uncomfortable pairing of those two facts is the whole of data governance in one example.

3. **The reply.** "No, and I want to be clear about why rather than just refusing. Storing card numbers moves us from the shortest PCI DSS questionnaire to the longest — from a handful of questions about keeping our site out of the way, to several hundred requirements and an external audit, on us, annually. That is a real recurring cost and a real liability, and it arrives the moment the first number is stored, not when something goes wrong. What our competitor is doing is almost certainly not what it looks like. The convenience you are describing is *tokenisation*: the provider stores the card and gives us a meaningless reference, so the customer does not retype anything and we never hold a number. Every provider we would use offers it, it is usually a configuration change rather than development, and it keeps the card where it is already somebody else's full-time job to protect. I can have that working this week." **If the client insisted:** say once more, in writing, what the consequence is and who carries it — the merchant, not the developer and not the provider. Then decline to implement it. This is not a matter of taste: it is a documented obligation the client would be taking on unknowingly, and building it after explaining that would make me a party to it rather than a contractor on it.

Chapter 23 — The Semester Project

1. **Five likely causes, in order.** (1) **A port already in use** on their machine — 8090 or 8091 is the commonest, and this handout hit it in Chapter 5. (2) **A step that lives only in your shell history**: a setting you changed once in the admin panel, a SQL statement you ran by hand, a file you copied. It works for you because you did it and forgot. (3) **Data that is not in the repository** — product images, a database dump — because the volume held it and the volume is not the repository. (4) **A line ending or path assumption**: an entrypoint script saved with CRLF will not run in the container, and the error is unhelpful. (5) **A different Docker version or no Docker at all. How to find out without their machine:** ask for three things — the exact command they ran, the complete output, and `docker compose ps`. Those three separate "it never started" from "it started and the shop is not installed" from

"it is running and they are at the wrong address", which is the whole diagnostic tree. **What should have been done in week 12:** build it from scratch on a machine that is not yours — `docker compose down -v` at minimum, a classmate's laptop ideally — because every one of the five causes is invisible on the machine where the shop was built and obvious on any other.

2. **What goes in the report:** both numbers, the index definition, the catalogue size, the machine, and the command that produces them — and then the honest sentence: the index made the page 8% slower, and here is what we think is happening. **What to investigate first:** whether the query is using the index at all. `EXPLAIN` before and after is one command and splits the result in two: if the optimiser ignored it, the question is why (a function on the column, as Chapter 6 found with `LCASE`, or a selectivity it did not like); if it used it and was slower, the index is being read in a different order from the table and the scattered reads cost more than the sort it saved — which is exactly what Chapter 6 measured at 50,000 products, twice. **Why it is worth more marks:** 15 ms is within noise for a single run, so the first thing it tests is whether you measured properly — repeats, a median, a warm cache. And if it survives that, it is a real result that contradicts the textbook rule, which is the kind of thing this course exists to teach you to notice. A result that went as expected demonstrates that you can follow instructions; this one demonstrates that you can measure.

3. **Three things still wrong with the shop as this handout left it.** (1) **The admin panel is still at `/admin/`** (Chapter 16, the one finding the hardening file could not close). *Cost:* an hour — rename the directory and the matching path in `admin/config.php`, then re-run the battery and tell whoever maintains it next. (2) **`system/storage/` is still inside the web root** (Chapter 16), blocked by a configuration file that any future administrator may replace during an upgrade. *Cost:* half a day — move it out, change the two constants in `config.php` and `admin/config.php`, and re-test uploads, sessions and the error log. (3) **There is no sitemap and no canonical link on any page** (Chapter 21). *Cost:* two or three days for a module following Chapter 19's structure, plus a one-line template override for the canonical, which is an afternoon and should be done first. **A fourth, if pressed:** the shop serves plain HTTP, so everything in Chapter 15 and Chapter 14 travels in clear — which is free to fix with a certificate and is the first thing to do before real customers, and is on Chapter 20's go-live list for that reason.